

Polynomial Time and Polynomial Space Algorithms for Empirically Testing Simple Normality On a Quantum Computer for Certain Constants

M.Chowdhury

Abstract

We give the first quantum algorithm that gives numerical evidence of if a constant is simply normal or not. Its a well known unsolved problem in number theory to determine if the digits of π are normal. The question of if the digits of π are normal (and other less well known constants) have fascinated non-mathematicians and mathematicians since the dawn of recorded history. The problem of proving π is normal in any base is a famous unsolved problem in number theory. The answer to this age old question (of why the digits appear of π appear random) is of interest to mathematicians and non-mathematicians. While our algorithm QBNTA are not in quantum polynomial time for π , based on known BBP formula for π , apart from π , they are the faster than their classical version giving a quite notable exponential speed up for the classic version of the algorithm (BNTA) for STC and other constants. It is polynomial in the number of qubits a and computes statistics for the first 2^a digits of STC in various bases. We give a polynomial time quantum computer program that can numerically verify if the digits of sums of Stoneham constants (which we also refer to as STC) is simply normal; the question of if the Stoneham type number $\alpha_{b,c_1} + \alpha_{b,c_2}$ (which is the sum of two Stoneham numbers is) simply b-normal was asked in [3]) The precise complexity of the algorithm is determined by the BBP formulae used. In the recent paper [3] the tested b-normality of $\alpha_{b,c_1} + \alpha_{b,c_2}$ classically. We use a modified Grover algorithm to obtain our result. We think our new algorithm is interesting because it is exponentially faster than the best classical algorithm for b-normal testing (BNTA). and the problem is not an artificial problem (e.g. like the Simon problem) but is as natural as something like integer factorisation when the constant is considered natural or fundamental.

1 Introduction

A BBP (Borwien-Bailey-Plouffe) formula [5] is an expression.

$$\sum_{n=1}^{\infty} \frac{p(n)}{b^{cn}q(n)} = c = BBP(c, b)$$

where $p(n)$ and $q(n)$ are polynomials in n with integer coefficients. The interesting property of the BBP formulae is that it allows an efficient digit extraction algorithm for individual c without calculation the previous digits using standard computer arithmetic implementation (i.e. does not need arbitrary precision implementation). Using the BBP formulae the d th digit can be computed in SC^* time in other words it uses $O(\log(d))$ space and $O(d \log d)$ time on a classical computer using the steps in the classical algorithm given in [5]. Suppose $p(n) = 1$. We now outline the BBP algorithm as described in [5]. Basically this is done by noting to compute the d th digit we need to compute

$$\begin{aligned} (b^d (\sum_{n=1}^{\infty} \frac{p(n)}{b^{cn}q(n)})) \bmod 1 &= \sum_{k=0}^{Floor(n/c)} \frac{b^{n-ck} \bmod p(k)}{p(k)} \bmod 1 + \sum_{k=Floor(n/c)}^{\infty} \frac{b^{n-ck}}{p(k)} \bmod 1 \\ &\text{we refer to } \sum_{k=0}^{Floor(n/c)} \frac{b^{n-ck} \bmod p(k)}{p(k)} \bmod 1 \text{ as the left half sum} \\ &\text{and } \sum_{k=Floor(n/c)}^{\infty} \frac{b^{n-ck}}{p(k)} \bmod 1 \text{ as the right half sum} \end{aligned}$$

for each term can be computed in the left half sum with the well known polynomial binary exponentiation algorithm [5], and in that left half of the sum floating point arithmetic is then used for the division and computer the sum mod 1. For the right half of the sum, where the exponent is negative, this sum converges rapidly, and we only need to compute a polynomial number of terms using floating point arithmetic for each term until the remaining terms are less than the machine "epsilon". The left half of the sum has $O(d)$ terms consequently this makes the complexity of the BBP algorithm for π in $O(\log(d)d)$. Full details of this algorithm are given in their seminal paper [5] but we outlined it here. The observation that we note that if our quantum oracle will use a BBP formula that has a polynomial number of terms using the classical BBP algorithm, then it turns out, as in the case for STC (Stoneham type number), this implies the quantum oracle can evaluate the above formula in polynomial time and space, our oracle is actually composition

of a BBP formula a constant number of times. Hence the complexity of the oracle is polynomial quantum time and polynomial quantum space.

Simply normal is defined as [2] if the occurrences of each of the B digits will appear with a limiting frequency of $\frac{1}{B}$ in the base B expansion of some constant c . No one has managed to prove that a naturally occurring number (such as π or e) is normal or even the simpler property that a naturally occurring number like π is simply normal-this is a remarkably difficult problem even for the case of simple normality. It is reasonable to say to prove even the simple normality of π is more difficult than proving a problem like the four colour theorem (which has been solved in 1976 after 125 years). The concept of normality was introduced in 1909 century by Borel. The best results related to that are the normality of artificially constructed numbers such as Champernowne constant [2]. c is said to be b -normal if every m -long string appears in the base b expansion of c with limiting frequency b^{-m} . It would be expected that to prove c is simply normal is easier to do than prove it is b normal.

In the recent paper [4] an empirical computation are done on a classical computer to determine statistically, if π is normal or not normal using their BNTA (b-Normal Testing Algorithm) algorithm. The algorithm 1 in [4] is the algorithm we refer to as BNTA. The only known algorithm to compute numerical evidence that a number is normal (for number such as STC or π) is works basically as follows. To compute each digit then make a count for each occurrence of the B digit, in the base B expansion is the BNTA given in [4], it computes the sample mean for each digit up to a known position in the expansion of c . In this paper we give a quantum version of a special case of BNTA (which we refer to as QBNTA)-this corresponds to $k = 1$ following the notation of [4], and $k = 1$ corresponds to string of length one or single characters or simple normality. This is the method used to confirm the normality of π on a classical computer see [4] where a world record computation of π was used for empirical evidence of its normality. Actually in [4] they used the Kondo and Lee world record computation for π which all the digits have to be computed up to to d th digit (which is not in polynomial space because all the digits up to the d th are stored) using the Chudnovsky formula [7]. Then a sub-string of the expansion of π is searched by "brute force" for every possible sub string, because BNTA is used, to gather evidence it is b -normal with this approach for simple normality, i.e. $k=1$, it takes $O(d)$ time and $O(d)$ space (i.e. more than 10 terabytes of space [6]) for up to the d th digit. This record computation was verified to be correct with a BBP formula.

In the case for the BBP formula for STC it turns out that the quan-

tum version of the algorithm gives an exponential speed up for the BNTA algorithm and the quantum algorithm is also of polynomial space and time complexity.

. Apart from the complexity comparison of the BNTA and QNBTA we think its interesting to show that a quantum computer can be used to get compute empirical numerical evidence that a constant is normal. This is the first application of quantum computers for normality testing . Our quantum algorithm is the also the best known in terms of time and space complexity for example if the BBP formula for Stoneham type constants of the form $\alpha_{b,c_1} + \alpha_{b,c_2}$ [3] is used- which has a polynomial time BBP algorithm. In other words for the constants of the form $\alpha_{b,c_1} + \alpha_{b,c_2}$ it gives an exponential speed up of the classical version of the algorithm which is BNTA. In [3] they do some numerical calculation to support if $\alpha_{b,c_1} + \alpha_{b,c_2}$ is b-normal (and hence simply normal).

To summarise this simple normality version of the known BNTA classical algorithm is as follows. $g(n, c, d, B)$ which is equal to 1 if in base B then nth digit of c is equal to d or is zero other wise.

$$\frac{1}{U} \sum_{n=1}^U \frac{g(n, c, d, B)}{n} \approx \frac{1}{B} \text{ for all } 0 \leq d \leq B-1 \text{ if } c \text{ is normal (or simply normal) in base } B$$

So BNTA (a classical algorithm) must evaluate each of the U terms to compute the statistical estimators (that have value approximately $\frac{1}{B}$ if c is normal) this method is used in [4] to give empirical evidence that c is normal or not empirical evidence c is not-normal. In this paper we give an exponential speed up of this algorithm using a quantum computer to this old number problem of number theory. Or to be more specific when $U = 2^a$ in the classical algorithm we must evaluate 2^a terms to compute a sample mean , in the quantum version of the algorithm the mean from the 2^a terms is computed as equivalently as one quantum computation. The other difference between the classical and quantum version are is: the quantum algorithm return the answer with some probability of error that can be made as small as desired and the classical version does not, the quantum algorithm only determines the decision if the above displayed equation is correct it does not actually give the count number for each digit. Since the many ages that have passed since the discovery of π it has been asked if the digits look random (i.e. are they normal). π was computed by Jon von Neumann on ENIAC in 1949 to get information about the distribution of π [4]. It would be interesting further work to see if the complexity of the our (QBNTA) quantum version of BNTA for simple normality can be improved for π . It turns out QNBTA is slower than the BNTA for π and we don't know if this situation can be improved.

Theorem 1 *Suppose c is a real number that has digit extraction formula; we may use BBP formula as the digit extraction algorithm here. Then there is a quantum algorithm (we refer to as QBNTA) that gives numerical statistics of the simple normality of for its binary expansion. This means using a known number qubits (a) it gives estimates of the of the proportion of zeroes or ones in the binary expansion of c up to a known position in the expansion. In other words it is a test for the simple normality of c .*

Write $f(n, c)$ for a function (here it is based on the BBP formulae in binary or a power of 2) that return the n th binary digit of the constant α . We will prove this by using GA (Grover's Algorithm) [8] to show that the mean value of $f(n, c)$ differs to the assumed value of $f(n, c)$. The new contribution in this paper is to apply GA to normal number testing, note we don't need to actually estimate to mean to some precision, as is done with classical BNTA, we need to decide empirically if its takes a certain value or not. We refer to an n such that $f(n, c) = 0$ is a bad solution. We refer to an n such that $f(n, c) = 1$ is a good solution.

Since c has a BBP formulae this implies that it has a classical digit extraction algorithm, which may be polynomial time or worse. Before GA is applied, the initial state of the input is.

$$I = \sum_{c=0}^{c=-1+2^a} \alpha_c |c\rangle$$

In other words we projected the initial state I in the Hilbert space of dimension 2^a . If w is the space spans the good solution w' are the bad solutions then this is

$$\frac{\sqrt{Z}}{\sqrt{2^a}} |w\rangle + \frac{\sqrt{-Z + 2^a}}{\sqrt{2^a}} |w'\rangle$$

into the two subspaces spanned by w and w' . I has some parts in are in the subspace $|w'\rangle$ and I has some parts in the subspace $|w\rangle$ and if it has an equal number of parts in each subspace the I makes as angle of $\frac{\pi}{4}$ with the coordinate system defined by w and w' . We interpret the angles between the the space spanned by the subspaces as a statistical estimator of how simply normal the real number c is. Now assume the number is normal. This means in the initial state the angle between the space spanned by the bad states (in GA this is n when $f(n) = 0$ is a bad state) and good states is close to $\frac{\pi}{2}$ radians because we assumed the number is normal to begin with. In the Grover iterating the angle between w and w' is hence

$$\sin^{-1}(\sqrt{M/N}) = \sin^{-1}(\sqrt{0.5N/N}) = \arcsin(\sqrt{1/2}) = \frac{1}{4}\pi \text{ radians}$$

as 2^a tends to infinity.

It is known number of Grover iterations k in the unmodified GA is and $1 < M < N$

$$k = \text{RoundInteger}\left(\frac{\pi}{4}\sqrt{\frac{N}{M}} - \frac{1}{2}\right)$$

We don't use this value of k above because we will use a modified GA an the above is for the standard GA . It is known in GA [8] that after k iterations it is known the angle between w' and w is nearest to $\pi/2$

$$(2k+1)\theta - \frac{\pi}{2}$$

gives the best success probability. But for our purposes we can select $k = 3$ (and other k will work) because that will leave the probability of observing a zero or one the same (50 percent chance) according to the Grover algorithm- because it will find an n that with 50% chance belongs to w or w' -so we wont use the number of iterations as suggested by Grover because we are using a modified Grover algorithm. Since the angle is then $\frac{7\pi}{2}$ and $\cos(\frac{7\pi}{4}) = \frac{1}{2}\sqrt{2} = \alpha_0$, $\sin(\frac{7\pi}{4}) = -\frac{1}{\sqrt{2}} = \alpha_1$, between the solution and the w' plane, would tend to these values because of a consequence of the central limit theorem as described below. This will actually rotate it past $\frac{\pi}{2}$ in the plane to a place that is approximately a good approximate reflection along a plane .

The key observation is that the amplitudes encode the mean frequency using information of the **all** the digits in the range from 0 to $-1 + 2^a$ (the global information) and these can be manipulated on a quantum computer with a polynomial number operations to get global information about the mean of the digits with a known probability (this observation also applies to a generalised version of this algorithm given after this proof)-in the classical version of the algorithm it requires an exponential number of steps to computer this global information (because each digit is computed individually e.g. with the Chudnovsky algorithm for digits of π) and on a quantum computer it takes a polynomial number of steps to compute this global information.

This suggest the following algorithm for testing normality of digits. Applying GA and measure $m = O(a)$ times, so do GA m times, for each application make a number of observations L_0 and L_1 observations to estimate the square of the values $\alpha_0 = \frac{\sqrt{Z}}{\sqrt{2^a}}$, $\alpha_1 = \frac{\sqrt{-Z+2^a}}{\sqrt{2^a}}$ which corresponds to measuring $|0\rangle$ or $|1\rangle$. To get a good estimate for α_0, α_1 using this simple method of repeated observation would require at least a linear time algorithm but the polynomial number $O(a)$ is good enough for our purposes to show if the digits are not normal. $O(a) = 10^4 a$ would be

a reasonable concrete choice because big confidence intervals below can be constructed. Actually can take $O(a) = Da$ for any constant $D > 0$ and the algorithm still uses a polynomial number of observations. Note we could adjust D to any value so that it corresponds to a high enough statistical confidence limit for the amplitudes to be computed to with a desired number of binary digits.

Hence if

$$\frac{L_0}{L_0 + L_1} = (\alpha'_0)^2 \text{ and } \frac{L_1}{L_0 + L_1} = (\alpha'_1)^2 \text{ are the estimates for the amplitudes}$$

differ significantly from $\frac{1}{2}$ we can conclude that is good quantum computer empirical evidence that suggest the c may not be normal.

But because we rotated to the point $(\cos(\frac{7\pi}{4}), \sin(\frac{7\pi}{4}))$, this simplifies the analysis, because we can consider $f(i, c)$ to behave like a coin toss, and use these well known statistics results about tossing a random coin. While generally different amplitudes correspond to the same probability here the probability is known to correspond to a single amplitude. Hence as a is increased the values $\frac{\sqrt{Z}}{\sqrt{2^a}}$ and $\frac{\sqrt{-Z+2^a}}{\sqrt{2^a}}$ both tend to $\frac{1}{\sqrt{2}}$ because it like a coin toss the amplitudes are the square of the sample means which do tend to the expected mean with error $O(\frac{1}{\sqrt{n}}) = O(\frac{1}{\sqrt{2^a}})$ this is a well known result which follows from the central limit theorem. It says that the standard deviation of the sample (i.e. our observations) is the same as the real standard deviation up to a factor $O(\frac{1}{\sqrt{\text{population size}}})$. Note the more qubits a we have the square of the amplitudes the better the approximation to the real mean $\frac{1}{2}$. Because the qubits can only make a rational approximation in their amplitude to the irrational $\frac{1}{\sqrt{2}}$ there will be some error because of this. We ignore this as it is considered a small error. We consider this as a small error because we assume we can build a quantum computer where a is large enough so that the error becomes small. As a rule of thumb the central limit theorem applies when the population size is 30 or more and so in our context it easily applies because $10^4 a > 30$. Hence if the observations do not agree with the central limit theorem, such as if the coin was fair what is the probability of observing a number of heads, we conclude that this is evidence that the number is not normal. If observations do agree with the central limit theorem we conclude this is evidence that the number is normal. Form the central limit theorem if we want to estimate the amplitude to an accuracy ϵ then we need to make $O(1/\epsilon^2)$ observations. For our purposes ϵ is fixed since we only require ϵ to be such that it is small enough to detect if the limit frequencies of the digits or different from $\frac{1}{2}$.

Note that this algorithm only works if there is a bias in the digit distribution that is detectable in $O(a)$ samples. For example if somewhere

in the expansion of c there it implies $\frac{L_0}{L_0+L_1} = \frac{227}{300}$ and $\frac{L_1}{L_0+L_1} = \frac{73}{300}$ then this is detectable from the sample size (because the first few binary digits of it are different from $\frac{1}{2}$). It could be argued for example that the algorithm returns the wrong answer since the observations to estimate α_0, α_1 e.g. it always return $|1\rangle$ for example and not a good estimate for the amplitudes. This is a source of probabilistic error in the algorithm- but in the real world- this unlikely to happen in practice. The probability that it return all $|1\rangle$ for example, when the number is normally distributed, is $(\frac{1}{2})^{10^4 a}$ which is quite small. To be precise this can be considered as the Bernoulli distribution (for a fair coin flip) and by the central limit theorem this is unlikely to happen and we can construct confidence intervals to show that the result is statistically reliable. We can consider the estimators $(\alpha'_0)^2$ and $(\alpha'_1)^2$ as unknown mean value estimates of an infinite population. Hence we use confidence intervals to determine how likely the estimators are in error of the actual real valued amplitudes. For a Bernoulli distribution there is a few choices for the confidence interval.

So the algorithm for empirical quantum normality testing (QBNTA) is as follows.

2 A Quantum Algorithm for Empirical Testing of Simple Normality

Quantum B-Normal Testing Algorithm (QBNTA)

Assume the number c is simply normal.

Step 4 and 5 are done by classical computations. The number of constant iteration are also computed classically-(no need for the usual exponential $O(\sqrt{N})$ times as explained above). Start with $H=0$. H is a counter for the number of Grover iterations. L_0, L_1 is set to zero initially are the counters if $f(n, c)$ is zero or one after measuring for n .

Step 1. Set the boolean $m = f(n, c)$ as the oracle in the Grover algorithm.

Step 2. Modified Grover algorithm

GA Step i. Initialise the register $|a\rangle$ to $|0^n\rangle$, and the work qubit w to $|1\rangle$

GA Step ii. Amplify $n+1$ Hadamard gates to get the equiprobable state equal to $I = \sum_{c=0}^{c=-1+2^a} \alpha_c |c\rangle$

GA Step iii) Apply the Grover iteration Q then increment the loop counter up to H

$$Q|a\rangle \rightarrow |a\rangle$$

Do the Grover iteration a constant number of times H

GA Step iv) Now measure register $|a\rangle$ to for the value of n .

Step 3. Measure the value for n . If $f(n) = 1$ increase L_1 by one otherwise increase L_0 by one.

Step 4. Repeat Steps 2 and 3 $O(\log(\text{"Size of population"})) = O(a) = Da$ times.

Step 5. The final step is to compute the estimators $(\alpha'_0)^2$ and $(\alpha'_1)^2$ and their confidence interval, if they significantly differ from $\frac{1}{2}$ then this is numerical evidence that the number may not be normal. Hence it is hoped it may be proved, with further work, a rigorous proof that the number is not normal. If the number may not be normal from the observed evidence then the assumption is false. End of algorithm.

Note that if the number c was not simply normal to begin, lets say, for example, the number of zeros become more dense as the position goes further in to the binary expansion of the number, then θ would tend to zero radians as the number of qubits a is increased, then rotating the bad states with $k = 3$ would not make much difference and there we have α'_0 tend to 1 and α'_1 tend to zero. The argument is similar if the limiting frequency of one of the digits tend to a small interval in $(0, 1)$ not near $(\frac{1}{2}, \frac{1}{2})$ and α'_1 may be non-zero in these cases. In all these cases, where the number is not simply normal, we selected the "wrong" number of iterations in the modified Grover algorithm, but selecting the "wrong" number of iteration is the right thing to do to detect if the number is simply normal. In this simpler version of QBNTA (we outline a generalised version below) we rotated to the point $(\cos(\frac{7\pi}{4}), \sin(\frac{7\pi}{4}))$ at a point in the plane, we started with the state I which was unmarked by the Grover iteration and if the number was normal by assumption then the probabilities observed would correspond to the point $(\cos(\frac{7\pi}{4}), \sin(\frac{7\pi}{4}))$. The reader may wrongly think that just measuring the initial state I we can deduce if the number is simply normal, but this is wrong because all we know is the initial angle the initial state GA (which is in the equiprob-

able state $\frac{1}{\sqrt{2^a}} \sum_{h=0}^{h=-1+2^a} |h\rangle$ that can be created using a Hadamard gates)

if it is assumed to be normal we have not yet used the oracle of GA to find the solution-if it turns out the assumption of normality was correct then consequently QBNTA rotates the solution to the final point $(\cos(\frac{7\pi}{4}), \sin(\frac{7\pi}{4}))$.

The actual oracle uses the modified BBP formula i.e.

$$f(n, c) = \text{Integerpart}(B * \text{FractionalPart}(B^n \text{BBP}(c, B)))$$

and is compute using the method described in the introduction for the

$FractionalPart(B^n BBP(c, B))$ part of the formulae. Its can be shown that a circuit for the above formula does exist.

End of proof.

3 A Sketch of a Generalised QBNTA

We can generalise this binary algorithm to different bases. So There is an another variant of this algorithm where the oracle is generalised to $g(n, c, d, B)$ is used instead of $f(n, c)$. Then if c is normal in base B we have a similar analysis to above but we sketch out the differences which are where

$$\begin{aligned} g(n, c, d, B) &= 1 \text{ if the } n\text{th digit of } c \text{ is equal to } d \text{ in base } B \\ &= 0 \text{ if the } n\text{th digit of } c \text{ is not equal to } d \text{ in base } B \\ &\text{where } d \in [0, B - 1] \text{ is the digit we are testing the frequency for} \end{aligned}$$

then in this variant if the number is normal in base B this gives the initial angle between the subspaces as

$$\sin^{-1}(\sqrt{M/N}) = \sin^{-1}(\sqrt{\frac{1}{B}N/N}) = \arcsin(\sqrt{1/B})$$

We sketch the algorithm as follows. By setting the generalised oracle to $g(n, c, d, B)$ which is equal to 1 if in base B then n th digit of c is equal to d or is zero other wise. We do $B - 1$ versions of the algorithm one for each digit. In each version the estimators $(\alpha'_0)^2$ and $(\alpha'_1)^2$ will be for the occurrence of d and the other amplitude is for the non occurrence of d . So for any value of B we only need to use GA a polynomial of times because the number of solution is of the form $\frac{N}{B}$ - in other words GA is a polynomial algorithm here. Because then the number of iterations is

$$k = RoundInteger(\frac{\pi}{4}\sqrt{\frac{N}{\frac{1}{B}N}} - \frac{1}{2}) = RoundInteger(\frac{\pi}{4}\sqrt{B} - \frac{1}{2})$$

So for the decimal we have

$$RoundInteger(\frac{\pi}{4}\sqrt{10} - \frac{1}{2}) = RoundInteger(1.9836) = 2$$

just one less iteration less than the binary case.

Note that we are not using a modified GA in this case of the generalised sketched algorithm. So we use the above known number of iterates

as suggested by Grover to get the probability close as possible to 100 percent.

Then to simplify things as $B \rightarrow \infty$ we would take the nearest integer such that

$$(2k+1)\theta - \frac{\pi}{2}$$

is close to zero and $(2k+1)\theta < \frac{\pi}{2}$ this is possible since as θ becomes smaller (i.e. $\lim_{B \rightarrow \infty} \arcsin(\sqrt{1/B}) = 0$) a better approximation to $\frac{\pi}{2}$ can be made the larger B is. Estimating α'_0 and α'_1 corresponds to observing a specific digit d in the expansion of c we would expect α'_0 to be near to 1 (and measured with with probability close to 1) and α'_1 to be near to 0 (and measured with with probability close to 0) when c is simply normal - if the estimators differ significantly ,e.g. if $\alpha'_1 \neq 0$, then its reasonable to say that is evidence that c may not be normal. For smaller values of B we don't get a good approximation to $\frac{\pi}{2}$ and so in these cases our algorithm may not work. To derive a base B^J digit extraction algorithm from a base BBP B algorithm we have

$$f(n, c) = \text{Integerpart}(B^J * \text{FractionalPart}(B^n \text{BBP}(c, B)))$$

instead. Since this generalised algorithm works to work better as $B \rightarrow \infty$ or equivalently some large enough $J > 1$. Or we could add in dummy qubits to increase the maximum size of the register and get a better approximation to $\frac{\pi}{2}$ and reduce the failure probability of GA.

For the number of observation, again from the central limit theorem, this is $O(\frac{1}{\epsilon^2}) = O(B^2)$, and for our purposes $\epsilon < \frac{1}{B}$ is fixed for a specific base, since we only require ϵ to be such that it is small enough to detect if the limit frequencies of the digits or different from $\frac{1}{B}$.

4 Application of QBNTA

In [1] an RNG (random number generator) which is presented. As it parameters it takes a BBP formulas where c is assumed to be normal (this is given in hypothesis A in [1]) and the generator is assumed to be random based on the unproven assumption of hypothesis A. An application of our new quantum algorithm would be to find inputs (i.e. it is a bad seed detector for this RNG) that may be bad for the RNG in [1] this would be done by taking the BBP formulae as input to QBNTA , if there is then empirical evidence that specific BBP formulae c is not normal it is not used for the RNG.

Another different application of QBNTA is as follows. It is not known if the sum of the two Stoneham constants [3] (defined in the next section)

$$\alpha_{b,c_1} + \alpha_{b,c_2}$$

is b-normal or simply normal. But as stated in [3] empirical analysis using their BNTA shows evidence it is b -normal, and our quantum algorithm can be applied to get evidence if $\alpha_{b,c_1} + \alpha_{b,c_2}$ is b normal. We give the oracle for Stoneham type numbers.

5 The Oracle for Stoneham Type Numbers

It is noted in [1] that the BBP algorithm in base b can be applied to the Stoneham numbers resulting in a polynomial time digit extraction algorithm using the BBP algorithm described in the introduction. In this section we give a proof that the oracle for Stoneham type constants is polynomial time and in polynomial space.

Stoneham defined the Stoneham number as

$$\alpha_{b,c} = \sum_{j=1}^{\infty} \frac{1}{c^j b^{c^j}}$$

b and c are coprime

we note the maximum value of the oracle for extracting the n th digit $-1 + 2^a$ digit. The motivation for this is it was asked in [3] the question of if the Stoneham type number

$$\alpha_{b,c_1} + \alpha_{b,c_2}, \text{ for } b \geq 2, c_1 \geq 2, c_2 \geq 2$$

b and c_1 are coprime, and
 b and c_2 are coprime

b -normal. The way we will construct the oracle, this is our polynomial implementation of the BBP algorithm, is we will construct the oracle using a truncated version of the above series to obtain a partial, and this same partial series will be used to extract a digit from 0th to $-1 + 2^a$ digit for the oracle. So it uses the same partial series with the digit as the parameter to extract that digit. We show that this partial series has a polynomial number of terms so if the series is evaluated term by term, and we use one polynomial space variable to store the result each term this means the oracle is polynomial space. So we need to find t in

$$b^{-1+2^a} \sum_{j=1}^t \frac{1}{c^j b^{c^j}}$$

so we can use the above to compute the $-1+2^a$ digit, i.e. what is the value of t for the series to converge enough, this will give the worst

case value for t because t becomes larger the bigger the position of the extracted digit is.

This is determined by the number of terms in the left half sum in the general BBP formulae given in the introduction. This is the minimal value of j such that $\frac{b^{-1+2^a}}{c^j b^{c^j}}$ is less than 1, it is given that the integer $c > 1$. Hence

$$\begin{aligned} c^j b^{c^j} < b^{-1+2^a} &\Rightarrow j \log c + c^j \log b < (-1 + 2^a) \ln b \\ &\Rightarrow c^j \log b < (-1 + 2^a) \ln b \Rightarrow c^j \log b < (2^a) \ln b \\ &\Rightarrow j \log(c \log b) < a \ln(2 \ln b) \Rightarrow j < a \ln(2 \ln b) \\ &\Rightarrow j = O(a) \end{aligned}$$

hence for the left hand side sum in the BBP formula $O(a)$ terms, which is a polynomial number of terms, will be sufficient which is polynomial in a . For the right half sum in the BBP algorithm we have $\frac{b^{-1+2^a}}{c^j b^{c^j}} < 1$ at t hence for all $j \geq t$ this can be crudely bounded above by the geometric progression with terms $\frac{1}{b^j}$ which will rapidly converge and show a polynomial number of terms are required to be accurate to within the machine epsilon, showing the right half sum in the BBP formula is also a polynomial number of terms s .

Hence

$$f''(n, c, b) = \text{Integerpart}(\left(\left(\sum_{j=1}^{t_1} \frac{b^{n-c^j} \bmod c^j}{c^j}\right) \bmod 1\right) + \left(\left(\sum_{j=1}^{t_1+s} \frac{b^n}{c^j b^{c^j}}\right) \bmod 1\right))$$

for some $t_1 < t$ which can be computed. $f''(n, c)$ gives the base b digit of $\alpha_{b,c}$ using

$$f(n, c, B) = \text{Integerpart}(B * \text{FractionalPart}(B^n \text{BBP}(c, B)))$$

Hence for the analysis is similar for constant $\alpha_{b,c_1} + \alpha_{b,c_2}$. Using the identity for any real numbers g and h $B^n \text{FractionalPart}(g + h) = \text{FractionalPart}(\text{FractionalPart}(B^n g) + \text{FractionalPart}(B^n h))$

we have

$$\begin{aligned} f(n, c, b) = &\text{Integerpart}(\left(b * \left(\left(\left(\left(\sum_{j=1}^{t_2} \frac{b^{n-c_1^j} \bmod c_1^j}{c_1^j}\right) \bmod 1\right) + \left(\sum_{j=1}^{t_2+s} \frac{b^n}{c_1^j b^{c_1^j}}\right) \bmod 1\right)\right) + \right. \\ &\left.\left(\left(\left(\sum_{j=1}^{t_3} \frac{b^{n-c_2^j} \bmod c_2^j}{c_2^j}\right) \bmod 1\right) + \left(\sum_{j=1}^{t_3+s} \frac{b^n}{c_2^j b^{c_2^j}}\right) \bmod 1\right)\right) \bmod 1) \end{aligned}$$

for some computable integers t_2 and t_3 which can be computed in polynomial time and space complexity and the parameters t_2 and t_3 grow polynomially, with the number of qubits, the further the digits are extracted from the start of the base b expansion. Hence we can construct a polynomial function $f(n, c, b)$ for the digits of $\alpha_{b,c_1} + \alpha_{b,c_2}$ that uses a polynomial number of quantum gates; for the binary version of our algorithm we take $b = 2$, and c_1 that is coprime with 2, c_2 that is coprime with 2. For the sketched generalised version of our algorithm we simply set b to the required base. $f(n, c, b)$ for STC is now shown to have an exact expression which would be used if the quantum computer was built along with the function $D_i(x)$ defined below-with a polynomial number of terms- where the oracle also computes t_2 and t_3 inside $f(n, c, b)$ and this results in a minimum of $O(a)$ terms in our BBP implementation which is the same as $O(a)$ terms as in the BBP implementation described in the introduction. The exact expression is as follows. Now define Heaviside step type function

$$\begin{aligned} u(x, y) &= 1 \text{ if } x < y \\ &= 0 \text{ if } x \geq y \end{aligned}$$

Hence we can express $f(n, c, b)$ with more terms (but still a polynomial number)

$$\begin{aligned} f(n, c, b) = & \text{Integerpart} \left(\right. & (1) \\ & b * \left(\left(\left(\left(\sum_{j=1}^{t_2+s} \frac{(b^{n-c_1^j} \bmod c_1^j) * u(c_1^j, n)}{c_1^j} \right) \bmod 1 \right) + \left(\sum_{j=1}^{t_2+s} \frac{b^n * u(n, c_1^j)}{c_1^j b^{c_1^j}} \right) \bmod 1 \right) \right) + \\ & \left. \left(\left(\left(\sum_{j=1}^{t_3+s} \frac{(b^{n-c_2^j} \bmod c_2^j) * u(c_2^j, n)}{c_2^j} \right) \bmod 1 \right) + \left(\sum_{j=1}^{t_3+s} \frac{b^n * u(n, c_2^j)}{c_2^j b^{c_2^j}} \right) \bmod 1 \right) \right) \bmod 1 \right) \end{aligned}$$

Define

$$\begin{aligned} D_i(x) &= 1 \text{ if } i = k \\ &= 0 \text{ if } i \neq k \end{aligned}$$

Then the oracle is

$$g(n, c, d, B) = D_d(f(n, c, B)).$$

So the quantum computer would be initialised by computing how many terms are required in the equation 1. We know that $j < a \ln(2 \ln b)$, obviously we could set $j = \text{Ceiling}(a \ln(2 \ln b))$ as that is still an upper

bound, and hence these are the maximal values of t_3 and t_2 . We know that the machine precision for a real number is up to a bits hence it is sufficient to compute $s = a$ since $b > 1$ because that makes the terms smaller than the machine epsilon in the right half sum of the term in the BBP algorithm. Hence we can set $t_2 + s = \text{Ceiling}(a \ln(2 \ln b)) + a$ which is a polynomial number of terms which is computed classically. So we can replace the upper limit in the four sums with $a(1 + \ln(2 \ln b))$ instead of $t_2 + s$ or $t_3 + s$. We can make a quantum circuit to evaluate the of these polynomial terms.

Practically we would not evaluate the maximum value (the maximum integer representable by the machine word which is $-1 + 2^a$) for the digit extraction, and this is not also done in the classical BBP algorithm [9], where they mention that $1.18 * 10^7 \approx 2^{24}$ is the limit (but they give no proof of this and mention that the maximum extracted digit grows roughly linear with d) for 64 bit arithmetic. see [9], and d is a minimum of $10^{14} \approx 2^{48}$ if 128 bit arithmetic is used see [9]. The reason, for using less than 128 bit length number or 48 bit length number is because there are subtle round off error in each term and each exact integer in the BBP algorithm (for any constant) in the left half sum must be exactly represented as an integer where using integer division. In our oracle since $j = O(a)$ then the numbers c_1^j, c_2^j in the oracle will approximately be the size of the largest word that can be represented on a bits hence logarithm efficiently and accurately represent these numbers. c_1^j grows like 2^j and consequently the maximum space need to accurately store it would be some fixed integer multiple of a bits; it would still be a polynomial algorithm in this case. Hence we would need to build an extra circuit that has more than a bits to accurately represent c_1^j . But we do not need to build this extra circuit because this is equivalent to scaling down instead which is done by computing a smaller maximal n th digit. So we can represent c_1^j using up to as bits for some $0 < s \leq 1$. So if our algorithm is implemented like the BBP algorithm described in those papers it means it has to use less than a qubits, for the maximal value of the extracted digit, and if it uses sa qubits, for some positive $s < 1$, this wont change the complexity of the algorithm. There is an easy way to modify our algorithm so it uses sa qubit where sa is an integer. For the π $24/64 = 0.375 = 48/128 = 0.375$ the value of s is 0.375 from the values given in [9]. We are not aware they give any proof that shows this value of s constant in [9]- they seem to have found this value empirically. Firstly lets describe a way to estimate s for our oracle, in the following paragraph, before we describe the easy modification.

The Stoneham BBP formula, in our implementation, has significantly less terms (a polynomial number compared to exponential) in than the

BBP π . We find the value of s empirically as is done in the classical version of the BBP π algorithm with a classical computer. We outline a possible method to estimate s for a quantum computer using a classical computer. We implement 64 bit arithmetic and compute $\alpha_{b,c_1} + \alpha_{b,c_2}$ digit by digit using the BBP algorithm method mentioned in the introduction. We then compute $\alpha_{b,c_1} + \alpha_{b,c_2}$ classically using 16 bit and 32 bit arithmetic digit by digit using our BBP implementation (used in the oracle) in parallel with the 64 bit computation. Hence the 64 bit computation will determine what maximal digit the 16 and 32 bit computations are correct up to. So in the 64 bit implementation we use the BBP algorithm method mentioned in the introduction from [5]. For the 16 and 32 bit computations we use the BBP algorithm used in the oracle which is described in this section. We then find by brute force comparison where their values start to differ in the expansions by comparing each of the digits with the 64 bit computation. Then we can find the value of s empirically there (the same way we computed it for π above) and use this for the value of s is correct for in QNBTA (in $f_{bounded}(n, c, b)$ described below). We can get a more accurate estimate for s by comparing computations in higher bits instead of 64 bit. So this is a reasonable heuristic way to prove the algorithm running time.

We now describe an easy modification of the oracle so it only works up to sa bits, in other words we calculate the statistics up to the $-1 + 2^{sa}$ digits, and for values greater than this it simulates the output as if the number was simply normal (even if it's not). In any case from the final result we can compute the actual normality statistics, without the simulated digit output, after sa bits. Hence the oracle $f_{bounded}(n, c, b)$, would be as follows

$$\begin{aligned} f_{bounded}(n, c, b) &= f(n, c, b) \text{ if } 1 \leq n \leq 2^{sa} \\ &= n \bmod b \text{ if } 2^{sa} < n \leq 2^a \end{aligned}$$

so when $b = 2$

$$\begin{aligned} f_{bounded}(n, c, b) &= f(n, c, b) \text{ if } 1 \leq n \leq 2^{sa} \\ &= n \bmod 2 \text{ if } 2^{sa} < n \leq 2^a \end{aligned}$$

The term $n \bmod 2$ would simulate the number being simply normal in binary and can be computed efficiently. Hence the observed mean would be

$$\begin{aligned} &\frac{2^{sa}}{2^a} \frac{1}{2} + \text{The frequency of the ones up to the } 2^{sa} \text{ position} = (\alpha'_1)^2 \\ &\frac{2^{sa}}{2^a} \left(1 - \frac{1}{2}\right) + \text{The non-frequency of the zeroes up to the } 2^{sa} \text{ position} = (\alpha'_0)^2 \end{aligned}$$

from which we can find the both the actual frequencies, i.e. using the frequency of the ones up to the 2^{sa} position. The analogy of this in the classical case of the BBP π algorithm is they use $s = 0.375$ as mentioned above which is analogous to some s value used in our oracle. This idea can be easily extended to an arbitrary base. This idea of just using the first block of digits can also be extended so we only do simple normality testing on a block of digits. Which means the oracle would use some lower bound for n instead of 1. This can be used to answer questions such as Brouwer's question [10] which is in the decimal expansion of π , is there a place where a thousand consecutive digits are all zero? The similar question for $\alpha_{b,c_1} + \alpha_{b,c_2}$ is, in the binary expansion of $\alpha_{b,c_1} + \alpha_{b,c_2}$, is there a place where a 2^E consecutive digits are all ones, for some $E \geq 1$? If the result has mean value 1 in the binary version of QBNTA then this will answer the question with high probability.

Also the value of s would depend specifically on how the oracle implements the BBP algorithm such as how multiple precision operations is implemented as is implied from [9].

There are other constants such as the Korobov type constants these are

$$\beta_{w,x,z} = \sum_{n=x^z, x^{z^2}, x^{z^3} \dots} \frac{1}{nw^n}$$

see [1] and are known to be b -normal [1] which would have a similar analysis to STC for our quantum oracle, and a more efficient polynomial time oracle for STC given above. To the best of our knowledge it is not known if

$$\beta_{b,x,z} + \alpha_{b,c_1} + \alpha_{b,c_2}$$

is b -normal. This seems harder to prove than proving if

$$\alpha_{b,c_1} + \alpha_{b,c_2}$$

is b -normal and it is not known if $\alpha_{b,c_1} + \alpha_{b,c_2}$ is b -normal [3]. We could derive a polynomial QBNTA for the above constant to test if it is b -normal.

In [1] the 10^{100} (googol) binary digit of $\alpha_{2,3}$ is computed and this is the deepest 1-bit computation for any constant (prior to that it was the 10^{15} th bit of π). Consequently a quantum computer, with a suitable register size, could make the deepest verification for any constant of simple normality of b -normality of certain constants.

Note that we do not need to restrict to using BBP formulae for extracting digits as there may be another algorithm unlike the BBP that the oracle can be a function of- which of course means that the analysis for s would not apply to the general version of the binary QBNTA.

6 A Quantum Algorithm for Testing if a Number is Rational

Theorem 2 *Suppose c is a real number that has efficient digit extraction formula; we may use BBP formula as the digit extraction algorithm here. Then there is an efficient quantum rationality testing algorithm (we refer to as QRNTA) that gives numerical evidence if the number is rational in some base. This means it detects if the expansion becomes periodic c up to a known position in the expansion. In other words it is a numerical test for the normality of c that says the number is not likely to be rational or irrational from the position of the digits of its base b expansion up to a certain point.*

Proof (Sketch). The new contribution of this algorithm is to apply quantum computing to efficient BBP algorithms to give a new algorithm to numerically test if a constant is rational or irrational. It is noted in [1] that the BBP algorithm for the Catalan constant, a fundamental constant, may be used to test if it is rational or irrational. We sketch out the algorithm in this section but do not analyse it rigorously. This algorithm can be used to test if

$$\alpha_{b,c_1} + \alpha_{b,c_2}$$

is rational. Hence this would imply that it is not normal in base b . If it is guessed that the number is rational with this algorithm then it is hoped this can be rigorously proved. Note that there is a BBP formula for Catalan's constant and it is an open problem to decide if this constant is rational or irrational. A number is rational in some base b if its expansion is periodic or terminates in some base b —we refer to this as a rational expansion. In other words it is known that if c is rational then there exists a period e such that

$$b^d c(b^e - 1) \in \mathbb{Z}.$$

So classically if assuming an upper bound for e we check the above for all possible values (which is an exponential classical computation in the worst case) to prove it is rational. There is no polynomial classical algorithm to find e as this can be shown to be a generalised version of the order finding problem used in Shor's algorithm. This algorithm solves a different problem to the Shor algorithm of course. This algorithm is a quantum version of this classical algorithm which uses our oracle based on a BBP formula. It works in polynomial efficiency because we can derive a polynomial time function for extracting digits of c . Which is coincidentally the oracle used in our normality testing algorithm. Assume

we want to test that the expansion of the number becomes a rational expansion after position d . The algorithm is as follows.

Step 1. Pick a random base b . This is a classical step. Using the BBP algorithm, outlined in the introduction, check on a classical computer that $(b^d(\sum_{n=1}^{\infty} \frac{p(n)}{b^{cn}q(n)})) \bmod 1$ is near an integer for some $d, d+1, \dots, d+a$. This means it is assumed the expansion terminates in base b and the number is rational. Otherwise assuming c is rational this must mean that either the period is 2 or more or the values of d used is too small. The next step assumes the value of d is still large enough so the period if its more than 1 this can be detected.

Step 2. We have a quantum computer with the register $|x\rangle|y\rangle$. Apply the transformation

$$|n, o\rangle \rightarrow |x, \sum_i P(R(n+i, c, b), i)\rangle$$

where

$R(n, c, B)$ the n th digit of c in base B

$P(x, i)$ is a function that takes a set of size i digits and maps (ideally injective) to an integer

Note the function P is required to ensure that the values that make up the period are unique enough. It is known the QFT will still detect the period if some of the values are the same (and its not likely to be another period). The upper bound for i here is polynomial. As a simple example we can the

$$\sum_i p(R(n, c, B), i) = \sum_{i=1}^{i=5} 2^{i-1} * R(n+i, c, B)$$

So here p is a weighted version of a number of digits nearby to the n th. It is assumed the period is two or greater so the first two terms on the RHS of above maybe non zero. For STC we take

$$R(n, c, B) = f(n, c) \text{ (equation one in base } B\text{)}.$$

This sets the state to

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle \sum_i P(R(n+i, c, b), i)\rangle$$

Step 3. Now measure the second register to observe k . Assume c is rational and we picked n and b so a ration expansion is detectable. Then the new state is

$$\Psi > = \frac{1}{\sqrt{M}} \sum_{a \in A} |a, k >$$

where A is the set of integer such that $a < q$ and $A\{u, u + e, u + 2e, u + (M - 1)e\}$. Where u is the value of the digit position (which we do not know) after the n that the expansion of c becomes periodic with period e . Thus M is of size approximate $\frac{q}{r}$ which is much greater than 1. hence

$$\Psi > = \frac{1}{\sqrt{M}} \sum_{w=0}^{M-1} |u + we, k >$$

Step 4. Apply the QFT to the second register. This collapses the entangled first register. Then

$$\begin{aligned} \Psi > &= \frac{1}{\sqrt{M}} \sum_{w=0}^{M-1} |u + we, k > \text{ becomes} \\ \Psi > &= \frac{1}{\sqrt{qM}} \sum_{v=0}^{q-1} \sum_{w=0}^{M-1} |\exp(2\pi i(u + we) * \frac{1}{q})|v, k > \\ &= \frac{1}{\sqrt{qM}} \sum_{v=0}^{q-1} e^{2\pi i v u / q} \sum_{w=0}^{M-1} |\exp(2\pi i(0 + we) * \frac{1}{q})|v, k > \\ &= \frac{1}{\sqrt{qM}} \sum_{v=0}^{q-1} e^{2\pi i v u / q} (\sum_{w=0}^{M-1} \Omega^d) |v, k > \end{aligned}$$

where $\Omega = e^{2\pi i w e / q}$ which is a root of unity.

Step 5. Measure the first register. Then register of we observer the state $|s >$ with probability

$$pr(s) = \frac{1}{Mq} |(\sum_{w=0}^{M-1} \Omega^d)|^2$$

Noting $\Omega = e^{2\pi i v e / q}$ is $\frac{ve}{q}$ is not close to an integer then power of Ω cancel out such that the observation is quite unlikely. In this case $(\sum_{w=0}^{M-1} \Omega^d)$ is a finite geometric progression with a small sum. If $\frac{ve}{q}$ is near

an integer then $pr(s) \approx \frac{M}{Nq} \approx \frac{1}{q}$ since $\Omega \approx 1$. Then the probability of observation is an integer such that

$$\frac{ve}{q} = L \text{ for some integer } L$$

Step 6. From the observed values can be found by an using an algorithm for continued fractions. The input to the algorithm if

$$\frac{\text{Observed Value}}{q} = \frac{v}{q}$$

computing the convergent of $\frac{v}{q}$ we have $\frac{a_i}{b_i}$. Then the values b_i . which are the denominators of the convergent, then the period e will be some integer multiple of b_i for some i . This can be found in polynomial time.

Step 7. This step requires classical computations only. If no e , in step 6, is found then our assumption that the number was rational is false; this would correspond to no period being observed, and the measurement and estimates for e would look random each time. This can be verified by computing $b^q c(b^{q+e} - 1)$, using the BBP series for c , which will not be an integer. Evidence the number may still be rational may be found if we use a quantum computer with a larger register. If a period e is found then this can be verified by computing $b^q c(b^{q+e} - 1)$, using the BBP series for c , which will be an integer. Also if a period e is found the result can be verified further, as likely the number is rational, by using a larger value of e . Hence this is evidence the number is rational and hence not b -normal.

7 Conclusion

The main result in this paper is the QBNTA algorithm. We considered the problem of empirically proving if some constant c is normal or not on a quantum computer. We have given a new quantum algorithm version which gave exponential speed up, for certain constants, for its classical version (i.e. the BNTA) that could empirically determine if c is simply normal , if there was significant enough bias of the digit distribution. Our algorithm is the first quantum algorithm in the area of normal numbers. Our algorithm can be used to detect bad quality seed RNGs for the algorithm given in [1]. Our algorithm returns the answer with error that can be made arbitrarily small sketched out a similar algorithm to our binary algorithm that is for an arbitrary base, that uses the standard Grover algorithm but with a non exponential number of iterations. The binary version of our algorithm is based on a modified Grover algorithm that takes a constant number of Grover iteration (not the usual $O(\sqrt{n})$ operations) and rotates so the solution is read out with a probability of

50% (not the usual near to 100% as possible)-and we note that using the central limit theorem, it takes a constant number of observations, to decide if its plausible that the number is normal or non-normal in some fixed base. We gave a specialisation of our algorithm for $\alpha_{b,c_1} + \alpha_{b,c_2}$ with the details of how to build an oracle- our algorithm is general and can apply to new digit extraction algorithm that may be discovered in the future. Our binary and sketched generalised algorithms is good for numbers with a base B expansion where the limiting frequency of one of the digits differ significantly from $\frac{1}{B}$. It is an open problem if the complexity for the implied QBNTA for π can be improved. It maybe asked if the idea for our algorithm can be applied to other number theory conjectures not related to normality-then it seems that the conjecture must be true for a conjectured positive proportion over some integer interval. E.g. it maybe possible to apply it to the zeroes of the Riemann Zeta function in some way.

8 References

- [1] D. Bailey and R. Crandall. Random Generators and Normal Numbers, Experimental Mathematics, Vol1, 2002
- [2] E. Weisstein, "Normal Number", from Mathworld, available at <http://mathworld.wolfram.com>
- [3] D. Bailey, J. Borwien Non-normality of Stoneham constants, The Ramanujan Journal, Volume 29, 2012
- [4] D. Bailey, J. Borwien, C. Claude, M. Dinneen, M. Dumitrescu, A. Yee, An Empirical Approach to the Normality of π , Experimental Mathematics, 2012
- [5] D. Bailey, P. Borwien, S. Plouffe, On the Rapid computation of various Poly logarithmic Constants, mathematics of Computation, vol 66, 1997
- [6] J. Yee, S. Kondo, 12.1 Trillion Digits of Pi, www.numberworld.org/misc_runs/pi-12t
- [7] Wikipedia Chudnovsky algorithm, http://en.wikipedia.org/wiki/Chudnovsky_algorithm
- [8] L. Grover, A Framework for fast Quantum Mechanical Algorithms, STOC '98
- [9] D. Bailey, The BBP algorithm for Pi, manuscript, www.davidbailey.com/dhbpapers
- [10] A History of Pi, Given as question 2 in the section "Open Question about the number π ", www.history.mcs.st-and.ac.uk/HistTopics/PI_through_the_ages.html